DOCUMENTO DE ARQUITETURA & DISCOVERY

My Meal 100% Offline

Como eliminar a dependência da nuvem com Reconhecimento de Voz e Processamento de Linguagem Natural (NLP/LLM) direto no dispositivo.

Outubro / 2026 Mobile-First SQLite & On-Device AI

1. O Ponto de Partida do My Meal

O **My Meal** já foi concebido sob o paradigma **Offline-First**:

- **Banco de Dados 100% Local**: Todas as tabelas (alimentos TACO, histórico de refeições, porções, metas) residem no `SQLite` nativo do aparelho.
- **Cálculos e Algoritmos Locais**: O recálculo de calorias, proporções (100g vs 1g) e o novo `MealPlanOptimizer` rodam inteiramente em código Dart no próprio dispositivo.

> 🎯 **Único Ponto de Dependência da Nuvem:**
> O envio do áudio gravado (`.m4a`) para a API Google Gemini Flash para fazer a transcrição e o parsing da refeição em formato JSON.

2. O Desafio: As Duas Camadas do Áudio

Hoje, o modelo multimodal do Gemini faz duas coisas simultâneas em um único disparo. Para rodar offline, o celular precisará realizar essas **duas tarefas em cadeia**:

1. **Reconhecimento de Fala (Speech-to-Text - STT)**
   Transforma as ondas sonoras da voz no texto correspondente em português brasileiro.
2. **Extração de Informação (NLU - Natural Language Understanding)**
   Lê o texto transcrito e extrai: Refeição, Alimentos, Quantidades numéricas e Porções associadas.

3. Camada 1: Reconhecimento de Fala Local (STT)

Para transcrever áudio em texto no celular sem internet, temos duas opções práticas de altíssima qualidade:

Opção A: Whisper On-Device (Tiny / Base) Recomendado

O modelo Whisper da OpenAI adaptado para dispositivos móveis via `whisper.cpp` ou `sherpa-onnx`.

- **Tamanho do Modelo**: ~40 MB (versão `tiny.en/pt` quantizada) ou ~80 MB (versão `base`).
- **Consumo de RAM**: ~120 MB a 160 MB durante o processamento.
- **Tempo de Resposta**: 1 a 2 segundos para áudios de 5 a 10 segundos em processadores modernos.
- **Grande Vantagem**: Totalmente autocontido. Roda exatamente igual em qualquer modelo de Android ou iPhone, sem depender do que o usuário tem instalado.

Opção B: STT Nativo do Sistema Operacional (Speech-to-Text)

Utilização das APIs de fala nativas através do pacote Flutter `speech_to_text` (Android SpeechRecognizer / iOS SFSpeechRecognizer).

- **Tamanho do Modelo**: **0 MB** adicionais no aplicativo.
- **Grande Vantagem**: Aproveita a NPU nativa do chip do celular e não aumenta o peso do download.
- **Ponto de Atenção**: No Android, depende de o usuário ter o pacote de reconhecimento offline em português brasileiro baixado no app Google do sistema.

4. Camada 2: Como Extrair os Alimentos do Texto?

Com o texto transcrito em mãos (ex: "*comi dois ovos médios e 50 gramas de tapioca*"), existem dois paradigmas bem diferentes para extrair a refeição:

Cenário 1: LLM Local no Celular (On-Device Small Language Model)

Rodar uma IA generativa compacta na memória do celular via `llama.cpp` ou `Google MediaPipe GenAI`.

- **Modelos mais eficientes**:
  - **Llama 3.2 1B (Meta)**: ~700 MB quantizado (Q4_K_M).
  - **Qwen 2.5 1.5B (Alibaba)**: ~1.0 GB quantizado (excelente em PT-BR).
  - **SmolLM2 360M**: ~220 MB quantizado.
- **Requisitos de Hardware**:
  - **RAM livre necessária**: 1.5 GB a 2.5 GB.
  - **Tempo de inferência**: 3 a 7 segundos em CPUs ARM64.

> ⚠️ **Alerta de Compatibilidade Mobile:**
> Em celulares intermediários/topo de linha (6GB a 12GB RAM) funciona perfeitamente. Em celulares de entrada (3GB ou 4GB de RAM), o Android frequentemente encerra o app por falta de memória (OOM Killer) ao tentar alocar 1.5GB para a LLM.

Cenário 2: Motor Algorítmico Inteligente em Dart (Sem LLM) Inovador & Rápido

Em 99% das vezes, frases de refeição seguem estruturas previsíveis ("*[quantidade] [unidade] de [alimento]*"). Um parser nativo em Dart construído especificamente para culinária brasileira:

- **Extrator Numérico**: Converte "dois", "duas", "meio", "1/2", "3" em valores numéricos.
- **Fuzzy Match Fonético**: O `FoodRepository` do My Meal já possui busca fonética com remoção de acentos e tratamento de plurais ("ovos" -> "ovo").
- **Mapeamento de Porções**: Cruza automaticamente medidas faladas ("médio", "fatia", "colher") com as porções cadastradas.

> 💡 **Vantagens do Parser Algorítmico:**
> • **0 MB** de download adicional.
> • Resposta em **menos de 5 milissegundos** (instantâneo).

> • **Zero alucinação** (nunca confunde números ou inventa alimentos).
> • Roda liso até no celular mais barato do mercado.

5. E a Função de Planejar Refeição com IA?

Na funcionalidade de **Planejar Refeição** (onde você fala o que pretende comer e o app calcula as porções para bater as proteínas e calorias do dia):

> ✅ **O Otimizador Já É 100% Offline!**
> A classe `MealPlanOptimizer` desenvolvida em Dart é puramente matemática e heurística. Ela roda localmente no aparelho em frações de milissegundo.

Ou seja: resolvendo a transcrição do áudio (STT), a inteligência de planejamento nutricional funciona **imediatamente sem qualquer necessidade de internet ou chamada de API**.

6. Tabela Comparativa de Arquiteturas

Critério	Gemini (Atual)	LLM Local (Llama 1B)	Whisper + Parser Dart
Conexão	Requer Internet	100% Offline	100% Offline
Download Extra	0 MB	+700 MB a 1.2 GB	+40 MB a 70 MB
Uso de RAM	~150 MB	1.8 GB a 2.5 GB	~200 MB
Latência	2 a 4s (Rede)	3 a 8s (CPU/NPU)	**< 1 segundo**
Celulares Suportados	100% (com web)	Apenas 6GB+ RAM	**100% dos celulares**
Custo de API	Pode ter limite/chave	**Zero / Gratuito**	**Zero / Gratuito**
Privacidade	Áudio vai para nuvem	**Fica no celular**	**Fica no celular**

7. Roadmap Recomendado de Execução

- **Fase 1: Motor Offline Inteligente em Dart (STT + Parser)**

  Implementar o reconhecimento de voz local (via Whisper Tiny ou STT nativo) acoplado a um `OfflineMealParser` em Dart. O aplicativo passa a funcionar 100% offline instantaneamente, sem aumentar o tamanho do app.

- **Fase 2: Unificação e Modo Fallback**

  O app tenta o processamento local primeiro. Se o usuário estiver conectado e desejar um detalhamento enciclopédico de pratos complexos, pode opcionalmente alternar para a nuvem.

- **Fase 3: Módulo Opcional de SLM Local (Llama 3.2 1B)**

  Para entusiastas de IA local com smartphones potentes, criar um botão em Ajustes: "*Baixar IA Generativa Local (700MB)*" via `flutter_llama_cpp` ou MediaPipe.